

Cortex Terminal V1

NSE Equity Trading Platform — Architecture & Data-Flow Reference

XGBoost + GRU ensemble (binary UP/DOWN, calibrated) · FinBERT news intelligence · Hawkes-kernel decay · combinatorial purged cross-validation with a deflated-Sharpe gate · multi-agent consensus · realistic NSE paper trading. This revision is reconstructed from the live source — the ML internals, fusion/consensus engines, schema, and worker loops are transcribed from code rather than summarised.

Python 3.11 · FastAPI

TimescaleDB pg16

Redis 7

XGBoost + GRU · ONNX/Treelite

FinBERT · Ollama

Next.js 16 · React 19

Prometheus + Grafana

Version 2.0 · 2026-06-01 · supersedes the v1.0 architecture document · production model **v1.1.x** · Classification: Confidential

CONTENTS

§1	System Overview & Technology Stack	§9	Pipeline 6 — Multi-Agent Consensus
§2	Infrastructure & Service Topology	§10	Pipeline 7 — Paper Trading Engine
§3	Data Architecture & Database Schema	§11	Pipeline 8 — CAI WebSocket Fan-out
§4	Pipeline 1 — Real-Time Market Data Feed	§12	Strategy Engine & Backtesting
§5	Pipeline 2 — Historical OHLCV Ingestion	§13	ML System Deep Dive
§6	Pipeline 3 — AI News Intelligence	§14	AI System Deep Dive
§7	Pipeline 4 — ML Prediction System	§15	Combined Confidence & Consensus Arithmetic
§8	Pipeline 5 — Signal Assembly & Fusion	§16	Safety, Kill Switch & Governance
		§17	Frontend & Charting Engine

§1 System Overview & Technology Stack

Cortex Terminal V1 is a production-grade AI trading terminal for the NSE (National Stock Exchange of India). It combines a real-time Upstox market feed, a dual ML ensemble (XGBoost + GRU), a FinBERT-powered news-intelligence layer, a multi-agent consensus engine, and a fully-costed paper-trading simulator. Two processes — an async FastAPI API and a separate asyncio worker — share TimescaleDB and Redis. Since v1.0 the platform has added a user-facing strategy engine, a fundamentals pipeline, TA-Lib candlestick pattern detection, and a substantially re-engineered ML subsystem.

LAYER	TECHNOLOGY	ROLE
API Server	Python 3.11 · FastAPI 0.115 · Uvicorn 0.32	Async REST + WebSocket; lifespan orchestration; loads ML models at startup
Worker Process	asyncio · 11 concurrent loops WAS 10	RSS ingestion, event processing, regime detection, drift, safety, data ingestion, heartbeat, correlation, suggestion expiry, cache invalidation, P&L
Database	TimescaleDB (PostgreSQL 16)	OHLCV/tick hypertables, AI event chain, ML registry, strategy & paper-trading ledgers, fundamentals
Cache / Pub-Sub	Redis 7 (redis-py 5.2 + hiredis)	LTP cache, signal TTL, kill-switch fast path, pub/sub for the CAI channels
ML Inference	XGBoost 2.0.3 (Treelite) + GRU (TF 2.21 → ONNX) · onnxruntime-gpu 1.19	Binary UP/DOWN ensemble with calibrated probabilities; ATR-based price levels
ML Training	Optuna 3.5 · scikit-learn 1.4 · in-house CPCV · evidently 0.4.33 NEW	Walk-forward training, hyperparameter search, deflated-Sharpe gate, drift monitoring
NLP	FinBERT (HuggingFace) · spaCy 3.7 · transformers 4.46	Sentiment scoring, named-entity recognition on financial news
LLM	Ollama llama3.1:8b → OpenAI GPT-4o → rules	Event-type classification via a three-level fallback chain
Technical / Patterns	TA-Lib (61 candlestick patterns) NEW	Native technical indicators and pattern detection
Frontend	Next.js 16 · React 19 · TanStack Query · lightweight-charts v5	Dashboard, Scanner, HawkEye radar, CAI dashboard, strategy & paper-trading UI
Monitoring	Prometheus 2.53 + Grafana 11.4	Signal rate, consensus latency, circuit-breaker states, P&L, worker-loop metrics
Broker / Data	Upstox API v3 (OAuth2 PKCE, protobuf WebSocket)	Live LTP feed, historical OHLCV, OAuth authentication

What changed since v1.0. The ML system moved from a 3-class BUY/SELL/HOLD softmax to a **binary UP/DOWN** classifier with ATR-normalised labelling and selective-classification HOLD (§7); ensemble weights are now Sharpe-optimised rather than fixed; probability calibration, combinatorial purged cross-validation, and a deflated-Sharpe gate were added (§13); and three new subsystems appeared — the strategy engine (§12), fundamentals ingestion, and TA-Lib pattern detection. Security, migrations, and the test suite were hardened (see the companion Technical Audit).

§2 Infrastructure & Service Topology

Services are orchestrated via Docker Compose (Kubernetes-compatible for production: TrustedHost middleware, readiness/liveness probes, a Prometheus /metrics endpoint). The API and worker run independently and communicate only through Postgres and Redis.

CONTAINER	PORT	PURPOSE	HEALTH
cortex-db — timescale/timescaledb-ha:pg16	5433	Primary store — OHLCV, ticks, AI events, ML registry, strategies, paper trades, fundamentals	pg_isready
cortex-redis — redis:7-alpine	6379	Cache, pub/sub, signal TTL, LTP throttle, kill-switch fast path	redis-cli ping
cortex-ollama — ollama/ollama	11434	Local LLM inference (llama3.1:8b) for event classification	/api/tags
cortex-api — FastAPI	8000	REST + WebSocket hub; ML models loaded into memory at startup	/health
cortex-frontend — Next.js	3000	SSR frontend + BFF proxy /api/v1/[...path]	implicit
cortex-prometheus — prom/prometheus 2.53	9090	Metrics scraping	—
cortex-grafana — grafana 11.4 NEW	3001	Dashboards over Prometheus	—

2.1 API startup sequence (lifespan)

Prometheus metrics → Redis pool (max 50, hiredis) → Upstox httpx pool (50 conns, 30s keepalive) → MarketFeedService (protobuf WS, per-instrument throttle) → DataIngestionService → **ML models loaded from the registry** (RegistryModelLoader → EnsemblePredictor; fatal in production if missing) → SignalScheduler (15-min batched cycle for Nifty 50 + Next 50) → SignalPipeline (event-driven on-demand) → CAI Redis listener (fans channels out to browser WebSockets) → paper-trading P&L worker. ML services are stored on app.state and injected into the scheduler — not re-instantiated per request.

2.2 Worker process — 11 concurrent loops

LOOP	CADENCE / TRIGGER	FUNCTION
rss_ingestion	300–900s + jitter	Poll financial RSS; SHA-256 dedup → ai_raw_events
event_processing	continuous	NLP → classify → credibility → fake-news → ai_event_classifications
regime_detection	scheduled	Per-symbol market regime → ai_regime_detections + Redis
drift_detection	~300s	Z-score + KS vs training baseline → demotion triggers
safety_monitoring	continuous	Volatility/frequency guards → ai_safety_triggers
data_ingestion	startup + hourly	OHLCV backfill + gap fill (\$5)
heartbeat	30s	Liveness timestamp to Redis
cache_invalidation	pub/sub	Invalidate list caches on new suggestions
suggestion_expiry	periodic	Expire trade suggestions past 24h; publish expiry events
correlation / consensus	event-driven	Multi-agent consensus (\$9)
paper-trading P&L	per LTP tick	Recompute open-position P&L → pnl:{portfolio}

§3 Data Architecture & Database Schema

The schema spans **53 tables** managed by a linear, forward-only chain of **39 Alembic migrations** (head 0039). Eight high-volume tables are TimescaleDB hypertables with explicit chunk, compression, and retention policies; the hypertable DDL is wrapped in a D0 block that degrades gracefully (a notice, not an error) when the extension is absent, so the schema also builds on vanilla Postgres.

3.1 Tables by domain (★ = TimescaleDB hypertable)

DOMAIN	TABLES
Market data (3)	instrument_master · upstox_ohlc★ · upstox_ticks★
ML system (8)	ml_model_metadata · unified_model_registry · ml_predictions★ · ml_drift_metrics★ · ml_features · ml_prediction_outcomes · ml_feedback_errors · ml_audit_logs
AI intelligence (13)	ai_raw_events★ · ai_processed_events★ · ai_nlp_results · ai_event_classifications · ai_source_credibility · ai_fake_news_flags · ai_ml_models · ai_drift_reports · ai_regime_detections★ · ai_active_strategies · ai_trading_signals★ · ai_kill_switches · ai_safety_triggers
Signals (2)	trade_suggestions · event_correlations
Strategy engine (6) NEW	strategies · strategy_activations · strategy_trades · strategy_backtest_runs · strategy_audit_logs · user_strategy_subscriptions
Paper trading (8)	portfolios · paper_orders · paper_fills · paper_positions · paper_position_conversions · paper_pnl_snapshots · paper_trade_outcomes · post_close_monitors
Fundamentals (8) NEW	company_fundamentals_profile · company_key_ratios · company_income_statement · company_balance_sheet · company_cash_flow · company_share_holdings · company_corporate_actions · company_competitors
Auth / user (5)	users · refresh_tokens · user_preferences · user_suggestion_compliance · watchlist_items

3.2 TimescaleDB hypertable policies

HYPERTABLE	CHUNK	COMPRESSION (AFTER)	RETENTION
upstox_ohlc	1 day	7 days (seg: instrument,timeframe)	365 days
upstox_ticks	1 day	1 day	7 days
ml_predictions	7 days	30 days	365 days
ml_drift_metrics	7 days	30 days	365 days
ai_raw_events	1 day	7 days	90 days
ai_processed_events	1 day	7 days	90 days
ai_regime_detections	7 days	30 days	—
ai_trading_signals	1 day	30 days	365 days

§4 Pipeline 1 — Real-Time Market Data Feed

Live prices flow from Upstox's protobuf WebSocket into Redis, throttled per-instrument to prevent saturation, and are consumed by the frontend, the scanner, the ML feature loader, and the paper-trading P&L worker.

STAGE	COMPONENT	DETAIL
OAuth2	UpstoxClient.set_access_token	OAuth2 PKCE; token held in-process on app.state, never returned to clients
WS connect	MarketFeedService	Opens wss://api.upstox.com/v3/feed/market-data-feed; subscribes to watchlist instrument keys
Decode	MarketDataFeedV3_pb2	Binary protobuf → dict (instrument_key, last_price, volume, ohlc, ltq, atp)
Throttle	_throttle()	Per-instrument last-publish vs MARKET_FEED_THROTTLE_MS (default 250 ms); faster frames dropped
Publish	PubSubClient.publish_json	Channel market:ltp:{key} — {price, volume, change_pct, ts}
LTP cache	Redis SET ltp:{key}	Latest price cached (TTL) for synchronous reads by scanner and the ML feature loader
Frontend	useLtp / useWebSocket	/api/v1/upstox/ws → React Query cache → LivePriceBadge re-renders
P&L worker	pnl_worker	Reads cached LTP → recomputes open-position P&L per tick → pnl:{portfolio_id}

§5 Pipeline 2 — Historical OHLCV Ingestion & Backfill

The data-ingestion worker populates the upstox_ohlcv hypertable with historical and ongoing candles — the primary source for ML features, technical indicators, and signal assembly.

STAGE	DETAIL
Startup backfill	On first run, computes each symbol's last stored candle; if gap > 0, chunks by date range. Rate limited to ~40 req/min (serial, concurrency 1) to stay under the Cloudflare burst threshold
Gap-check loop	Every DATA_INGESTION_CHECK_INTERVAL (1 h), re-queries each symbol's latest timestamp; missing/stale records re-fetched. Max 3 retries → DLQ
Storage	upstox_ohlcv (instrument_key, timeframe, timestamp, OHLC, volume); bulk upsert via INSERT ... ON CONFLICT DO NOTHING
Timeframes	1D · 1week · 1hour · 30m · 15m · 5m · 1m, per symbol universe
Universe	NSE equity instruments via instrument_master, loaded from the Upstox instruments gzip (streamed with ijson)
Rate limiting	httpx with Retry-After parsing on 429; exponential backoff + jitter; circuit breaker (threshold 5, 60s timeout)

§6 Pipeline 3 — AI News Intelligence

A six-stage assembly line converting raw financial news into structured event classifications with impact scores, per-event decay parameters, and affected-symbol mappings. Runs continuously in the worker.

STAGE	COMPONENT	OUTPUT / DETAIL
1. RSS fetch	rss_fetcher	Poll feeds (300–900s + jitter, concurrency 5); SHA-256 content-hash dedup → ai_raw_events
2. Pre-process	event_parser	Strip HTML, normalise encoding, detect language; non-English translated via Ollama → ai_processed_events
3. NLP	nlp_engine	FinBERT sentiment ∈ [-1,+1]; spaCy NER (companies/people/locations); confidence ∈ [0,1] → ai_nlp_results
4. Classify	event_classifier · llm_client	Three-level fallback: Ollama llama3.1:8b → (conf<0.7) GPT-4o → (conf<0.5) rules. Outputs event_type, impact ∈ [-100,+100], affected_symbols[], decay half-lives → ai_event_classifications
5. Credibility	credibility_scorer	score = confirmed/total × 100; new sources start at 50; low-credibility impact dampened pre-fusion
6. Fake news	fake_news_detector	Cross-source consistency + historical accuracy + LLM plausibility; flagged events excluded from signal assembly

6.1 Decay half-lives by event type (stored per event)

The classifier writes decay_half_life_hours (fast) and decay_slow_half_life_hours per event; the fusion engine reads them back for the Hawkes-kernel decay (§8/§14).

EVENT TYPE	FAST (H)	SLOW (H)	INTERPRETATION
earnings	12	72	Intraday reaction ~12h; fundamental repricing ~3 days
fed_announcement	8	48	Immediate spike + ~2-day carry
merger_acquisition	12	120	Spike + long-tail deal uncertainty (~5 days)

EVENT TYPE	FAST (H)	SLOW (H)	INTERPRETATION
regulatory	24	168	Slow absorption, week-long structural impact
geopolitical	8	72	Sharp spike + 3-day carry
market_data	6	24	Priced within one session
company_news / general	12	48	~2-day relevance window (default fallback)
sector_news	18	72	Broader, slower absorption

§7 Pipeline 4 — ML Prediction System

The ML subsystem is a dual-model ensemble (XGBoost + GRU) trained offline on ~10 years of NSE OHLCV, serialised to ONNX/Treelite, and served from the registry. Versus v1.0 the label scheme, calibration, and ensemble weighting all changed; this section transcribes the current behaviour.

7.1 Label generation — binary UP/DOWN with an ATR dead-zone CHANGED FROM 3-CLASS

Targets are binary, with a symmetric ATR-normalised "dead zone" that excludes near-zero moves from *both* sides to prevent a "always predict DOWN" bias. HOLD is **not** a trained class — it is produced at inference by a confidence threshold on P(UP), i.e. *selective classification*.

```
τ = clip( atr_multiplier · ATR14 / price , min=0.001 )      # default atr_multiplier = 0.5 → ~0.5% band; fixed 0.5% fallback
horizon h = 5 bars

future_return > +τ → 1 (UP)
future_return < -τ → 0 (DOWN)
|future_return| ≤ τ → NaN (dead zone – dropped from training)
```

7.2 Feature engineering — 69 features

The feature store registers ~69 features per symbol/timeframe. Technical groups (confirmed from `ml/config.py`): momentum (15) — RSI-14/21, MACD line/signal/hist, Stochastic %K/%D, ROC-10/20, Williams %R, CCI-20, Momentum-10, TSI, Ultimate & Awesome oscillators; trend (12) — SMA-20/50/200, EMA-12/26/20/50, EMA-crossover flag, DEMA/TEMA-20, ADX-14, Aroon-up; volatility (8) — ATR-14, Bollinger upper/mid/lower/width, Keltner upper/lower, historical vol; volume (5) — OBV, volume-SMA-20, VWAP, MFI-14, A/D line; market structure (4) — support, resistance, pivot, Fibonacci. These are complemented by **fundamental features** (from the fundamentals pipeline) and **sentiment features** (from the AI layer) to reach the full set.

7.3 Ensemble inference & calibration

```
# 1. Per-model probabilities (binary)
P_xgb = XGBoost(features)      # Treelite-compiled, fast tabular inference
P_gru = GRU(sequence[60 × 8])  # ONNX runtime (GPU)

# 2. Post-hoc calibration (fitted on CPCV out-of-fold preds, ECE-checked @ 0.05)
P_xgbcal = sigmoid( a·log(s) - b·log(1-s) + c )  # XGBoost: Beta calibration (Kull 2017)
P_grucal = softmax( logits / T )                # GRU: temperature scaling (Guo 2017)

# 3. Weighted ensemble – weights learned by Sharpe optimisation, not fixed
P_ens = w_xgb · P_xgbcal + w_gru · P_grucal
      # trained weights ≈ (0.51, 0.49); constructor default (0.75, 0.25); XGBoost-only fallback if GRU absent

# 4. Direction – selective classification
direction = UP    if P_ens(UP) ≥ θ_conf
            DOWN  if P_ens(DOWN) ≥ θ_conf
            HOLD  otherwise          # low-confidence abstain
confidence = max( P_ens )
```

7.4 Price levels & caching

```
entry = current_ltp (Redis LTP cache)
SL     = entry ∓ k_SL · ATR14          # ATR-based; multipliers configured in _post_process (default ≈ 1.5)
TP1/2/3 = entry ± (2.0 / 3.0 / 4.5) · ATR14  # R:R ≈ 1.3 / 2.0 / 3.0
cache  : Redis SETEX ml:pred:{symbol}:{tf} TTL = SIGNAL_ON_DEMAND_CACHE_TTL
```

§8 Pipeline 5 — Signal Assembly & Fusion

The SignalAssembler fuses three independent sources — AI event intelligence, the ML ensemble, and a technical signal — into one directional signal in `ai_trading_signals`. The constructor now defaults to the Phase 5+ weights, with technical wired in.

PHASE	EVENT	ML	TECHNICAL	STATUS
Phase 2–4 (legacy)	0.50	0.50	0.00	Technical was a stub
Phase 5+ (current default)	0.35	0.40	0.25	TECHNICAL NOW WIRED — validated to sum to 1.0 at construction

```
# Source 1 – AI event signal (Hawkes-kernel decay, 48h lookback)
decay(t) = 0.7*0.5^(t/fast_hl) + 0.3*0.5^(t/slow_hl)      # t = event age (h); half-lives stored per event ($6.1)
score_i  = impact_i * decay(t_i)
event_score = clip( Σ score_i , -100, +100 ) / √N        # √N dampening: many small events ≠ one big event
event_conf = ( Σ classification_conf_i * decay(t_i) ) / N

# Source 2 – technical signal (Wilder RSI-14 + EMA-20/50 crossover)
RSI>60 AND EMA20>EMA50 → +100 | RSI>55 OR EMA20>EMA50 → +50
RSI<40 AND EMA20# Source 3 – ML signal ($7): ml_score = +100 UP / -100 DOWN / 0 HOLD ; ml_conf = P_ens(direction)

# Fusion – renormalise over available sources, then weighted average
D = Σ w_s (available only)      w*_s = w_s / D
fused_score = Σ w*_s * score_s   fused_conf = Σ w*_s * conf_s
if only one source available: fused_conf = min(fused_conf, 0.65)    # single-source cap

direction = BUY if fused_score > +50 ; SELL if < -50 ; else HOLD
confidence band: ≥ 0.80 high | ≥ 0.60 medium | ≥ 0 low
```

§9 Pipeline 6 — Multi-Agent Consensus & Trade Suggestions

The EventCorrelationEngine runs a bidirectional, two-pathway consensus. Three agents — Scanner (0.30), AI (0.40), ML (0.30) — must agree on direction before a suggestion is created. Target: sub-100ms p95 consensus latency.

PATHWAY	FLOW
1 · Technical-first	Scanner anomaly (volume ≥ 2× & RSI/price threshold) → parallel gather_event_signals + gather_ml_signals (5s timeout) → consensus → trade_suggestions + event_correlations
2 · Fundamental-first	High-impact AI event → for each affected_symbol: gather scanner state + ML prediction (5s/symbol) → consensus → list of suggestions

```
# Hard directional alignment gate (must pass before scoring)
require scanner_dir == ai_dir == ml_dir ∈ {BUY, SELL}
ML HOLD → immediate reject | any mismatch → DIRECTION_MISMATCH, return None

# Weighted consensus (confidences normalised to [0,100])
consensus = 0.30*scanner_conf + 0.40*(|ai_conf|*100) + 0.30*(ml_conf*100)
HIGH if consensus ≥ 80 → TradeSuggestion (confidence_level = HIGH)
MEDIUM if consensus ≥ 60 → TradeSuggestion (confidence_level = MEDIUM)
else → discarded (LOW_CONFIDENCE in event_correlations)

R:R = |TP1 - entry| / |entry - SL|      expires_at = generated_at + 24h
```

Per-agent **circuit breakers** (failure threshold 5, 60s timeout) bypass a failing agent and redistribute weight. A live suggestion is **re-evaluated** when consensus changes by ≥ 5 points or entry price by ≥ 2% (DRASTIC_SCORE_DELTA / DRASTIC_PRICE_DELTA_PCT).

§10 Pipeline 7 — Paper Trading Engine

A realistic NSE simulation with risk-based sizing, full statutory charges, and tick-level P&L. All money math uses Python Decimal with ROUND_HALF_UP to avoid float drift.

```
# Kelly-style position sizing
capital_at_risk = current_cash * (risk_per_trade_pct / 100)
stop_distance   = |entry - stop_loss| (fallback: entry * 0.01 if no SL)
raw_qty         = capital_at_risk / stop_distance
suggested_qty   = clamp( floor(raw_qty), 1, floor(current_cash / entry) )
```

CHARGE (NSE, FY 2025-26)	RATE	APPLIED TO
STT — CNC delivery	0.100%	Both sides (buy + sell)
STT — MIS intraday	0.025%	Sell side only
NSE exchange charge	0.00297%	Each side

CHARGE (NSE, FY 2025-26)	RATE	APPLIED TO
SEBI turnover charge	₹10 / crore	Both sides
GST	18%	On (exchange + SEBI) charges
Stamp duty — CNC buy / MIS buy	0.015% / 0.003%	Buy side only

```
gross_pnl = (current_ltp - entry) * qty          # long
net_pnl   = gross_pnl - charges_open - charges_close
pnl_pct   = net_pnl / (entry * qty) * 100        # published per tick → pnl:{portfolio_id}
```

Additional tables track position conversions (MIS ↔ CNC), P&L snapshots, realised outcomes, and post-close monitoring of positions held overnight.

§11 Pipeline 8 — CAI Real-Time WebSocket Fan-out

A single `cai_redis_listener` task subscribes to Redis pub/sub channels and fans messages out to in-process browser WebSocket clients.

CHANNEL	FRONTEND HOOK	CONTENT
<code>cai:signals:all</code>	<code>useSignalsRealtime</code>	Full <code>AITradingSignal</code> JSON — symbol, action, confidence, levels, contributing events
<code>cai:signals:{symbol}</code>	<code>useSignalsRealtime</code>	Per-symbol fan-out
<code>cai:regime:{symbol}</code>	<code>useRegime</code>	Regime type, volatility index, trend strength
<code>cai:models:updates</code>	<code>useModelsRealtime</code>	Model state transitions, deployment events
<code>cai:governance:alerts</code>	<code>MLModelsPanel</code>	Drift results, accuracy drops, demotion triggers
<code>suggestions:new</code>	<code>useSignals (HawkEye)</code>	<code>suggestion_id</code> → frontend re-fetches via REST
<code>pnl:{portfolio_id}</code>	<code>usePnlWebSocket</code>	Position-level net/gross P&L per tick

§12 Strategy Engine & Backtesting NEW SINCE V1.0

A user-facing subsystem (`services/strategy_engine/`) lets users define rule-based strategies, backtest them, and activate them against the live signal stream. Not present in v1.0.

COMPONENT	ROLE
<code>rule_evaluator</code>	Evaluates a strategy's condition tree against market context and indicators
<code>filter_pipeline</code>	Multi-stage filter (universe, liquidity, regime) before rule evaluation
<code>market_context</code>	Per-symbol context (price, EMA-20 signal, volume, regime) shared with the rule evaluator
<code>backtest_engine / backtest_runner</code>	Historical replay over <code>upstox_ohlcv</code> → <code>strategy_backtest_runs</code> + <code>strategy_trades</code>
<code>activation_fsm</code>	Finite-state machine governing a strategy's lifecycle (draft → active → paused) in <code>strategy_activations</code>
<code>dispatcher / sl_tp_worker</code>	Dispatches activated strategies; a dedicated worker monitors SL/TP on strategy positions

Backed by six tables (§3) plus `strategy_audit_logs` and `user_strategy_subscriptions`; exposed via admin and user APIs and a frontend strategy-builder / backtests UI.

§13 ML System Deep Dive — Training & Validation

Models are trained offline, validated with combinatorial purged cross-validation, gated on a deflated Sharpe ratio, calibrated, and registered through a human-gated promotion lifecycle. The latest committed run (2026-06-01) produced production model **v1.1.x**.

13.1 Combinatorial Purged Cross-Validation (in-house)

Implemented dependency-free (not `skfolio`/`mlfinlab`) because `skfolio` forces `numpy≥2` / `sklearn≥1.6`, which breaks the TF-2.21-GPU stack. Panel (multi-symbol) data is grouped on the global unique-timestamp axis; purge and embargo are computed there.

```
groups n = 8 , test groups = 2 , embargo = 5 , label horizon h = 5
n_splits = C(8,2) = 28          n_paths = n_test * C(n_groups, n_test) / n_groups = 2 * 28 / 8 = 7
purge : drop train t if its label window [t, t+h] overlaps any test block
embargo: drop train t within 5 steps after a test block
```

13.2 Deflated Sharpe gate (Bailey & López de Prado, 2014)

```
var_factor = 1 - skew·SR + (kurt_raw - 1)/4 · SR²
DSR        = Φ( (SR - SR₀)·√(T-1) / √var_factor )      SR₀ from multiple-testing correction (N trials)
reject if   DSR < 0.95                                # textbook threshold
PBO        = fraction of CPCV paths with non-positive net-of-cost Sharpe  (≥ 0.5 ⇒ likely overfit)
```

13.3 Pipeline & latest run (2026-06-01)

STAGE	DETAIL
Data	~10y NSE OHLCV (2015–2025); 2,011 symbols used (of 2,551 requested, after coverage filtering ≥ 0.6); ~3.76M samples; daily + weekly frames
HPO	Optuna (XGBoost ~100 trials; GRU fewer) + keras-tuner for GRU architecture
Calibration	Beta (XGBoost) / temperature (GRU) on CPCV out-of-fold predictions; ECE-checked
Ensemble	Sharpe-optimised weights ≈ 0.51 / 0.49
Gate	CPCV + deflated Sharpe + PBO
Registry	Versioned artifacts (XGBoost JSON, GRU keras + ONNX) + unified_model_registry; promotion shadow → paper/staging → live (human-gated)
Drift	Every ~300s: Z-score (≥ 2σ) + KS (p < 0.05) vs training baseline; accuracy drop > 5% → demotion

```
# Latest-run held-out metrics (30,000 samples)
          accuracy  Sharpe
XGBoost    0.640    0.97
GRU         0.664    1.02
ENSEMBLE    0.680    1.09    deflated Sharpe = 0.73    (ensemble beats both standalones)
```

Governance note — deflated-Sharpe threshold. The committed v1.1.x model reports a deflated Sharpe of **0.73**, below the textbook rejection threshold of **0.95**, yet it was promoted. Either the configured gate threshold sits below 0.95 or promotion uses additional criteria (e.g. raw Sharpe + PBO). This is worth confirming and documenting explicitly in the promotion config; the value is reported here as measured.

§14 AI System Deep Dive — NLP, Classification & Decay

The AI subsystem operates independently of ML until fusion. Financial text passes through sentiment, NER, classification, credibility, and fake-news stages, each contributing to a decayed, credibility-adjusted event signal.

```
# 1. Per-event classification confidence
classification_conf = P(event_type | text, entities) ∈ [0,1]

# 2. Credibility-adjusted impact
adjusted_impact = impact_score · (credibility_score / 100)

# 3. Fake-news dampening
flag == 'flagged'    → event excluded from signal assembly entirely
flag == 'suspicious' → impact_score · 0.3          (70% dampening)

# 4. Hawkes-decayed aggregate (entering fusion)
decay_i    = 0.7·0.5^(t_i/fast_hl) + 0.3·0.5^(t_i/slow_hl)
event_score = clip( Σ adjusted_impact_i · decay_i , -100, +100 ) / √N
event_conf  = ( Σ classification_conf_i · decay_i ) / N
```

§15 Combined Confidence & Consensus Arithmetic

End-to-end, from raw inputs to a consensus-gated trade suggestion. The ML branch is now binary and calibrated; the AI branch is Hawkes-decayed; the technical branch is RSI/EMA. Fusion renormalises over available sources; consensus applies a hard alignment gate.

ML BRANCH

AI BRANCH

TECH BRANCH

FUSION

CONSENSUS

P_ens = w_xgb·P_xgb^{cal} + w_gru·P_gru^{cal} → direction (selective), ml_conf = max P_ens

event_score = clip(Σ impact·cred·decay, ±100)/√N ; event_conf = Σ(conf·decay)/N

RSI-14 + EMA-20/50 → tech_score ∈ {-100,-50,0,+50,+100} ; tech_conf = |RSI-50|/50

D = Σ active weights ; w* = w/D # weights: event 0.35 / ml 0.40 / tech 0.25

fused_score = Σ w*·score ; fused_conf = Σ w*·conf

single-source cap: fused_conf ≤ 0.65

direction = BUY if >+50 / SELL if <-50 / else HOLD

gate: scanner_dir == ai_dir == ml_dir ∈ {BUY,SELL} # ML HOLD → reject

consensus = 0.30·scanner + 0.40·ai + 0.30·ml (conf ×100)

HIGH ≥ 80 / MEDIUM ≥ 60 / else reject → TradeSuggestion (expires +24h)

§16 Safety, Kill Switch & Governance

A layered safety system halts or degrades signal generation under abnormal conditions. The global kill-switch check is on the hot path of every signal and is Redis-cached to stay under ~2ms.

MECHANISM	TRIGGER	ACTION
Global kill switch	Manual OR loss_rate > 5%	All signal generation halted; pipeline returns skipped
Instrument kill switch	Per-symbol anomaly or manual	Signals skipped for that symbol only
Volatility spike	Volatility > 3× normal	Signals paused; AISafetyTrigger recorded; auto-resume on normalisation
Signal-frequency guard	rate > 100/min	Kill switch activated (runaway-loop protection)
Circuit breakers (ML / AI / Scanner)	5 consecutive failures / 60s	Agent bypassed, weight redistributed; HALF_OPEN test after 60s; close on first success
Model drift demotion	Z > 2σ or accuracy drop > 5%	live → paper → shadow; cai:governance:alerts published

Fast path: KillSwitchManager.is_active(global) reads Redis key kill_switch:global (TTL-cached, <1ms); on cache miss it falls back to a ai_kill_switches SELECT and re-primers Redis — keeping the check under ~2ms on the hot signal path.

§17 Frontend & Charting Engine

The Next.js frontend uses a BFF catch-all proxy (/api/v1/[...path]) that injects auth, plus direct WebSocket connections for real-time streams. JWT auth uses HttpOnly cookies with refresh rotation; WS connections validate a token query param.

PAGE	HOOKS	WEBSOCKET CHANNELS
/ (Dashboard)	useWatchlist · useLtp · usePaperTrading	market:ltp · pnl:{portfolio}
/hawk-eye-radar	useSignals · useTradeSuggestions	suggestions:new
/scanner	useScanStream	SSE /api/v1/scanner/stream
/cai-dashboard	useCAIWebSocket · useRegime · useModelsRealtime	cai:signals:all · cai:regime · cai:models
/stocks/[symbol]	useChart · useLtp · useMLAnalysis	market:ltp
/admin/governance	useModels · useModelsRealtime	cai:models · cai:governance:alerts

17.1 Charting engine (lightweight-charts v5, 100% client-side)

A DB-first CandleService (TimescaleDB → Upstox fallback) feeds zero-dependency native-TypeScript indicators; a shared useChart() hook manages the chart lifecycle. Nine indicators across overlays and oscillators:

TYPE	INDICATORS
Overlays	SMA-20/50 · EMA-20/50 · Bollinger (20,2) · VWAP (daily reset, IST)
Oscillators	RSI-14 (Wilder) · MACD (12,26,9) · Stochastic (14,3,3 slow)

A pane-layout engine allocates oscillator zones dynamically (OSCILLATOR_ZONE = 0.20, VOLUME_RATIO = 0.22); live ticks update the last candle in a useMemo so indicators are not recomputed every tick; infinite-scroll history loads on a visible-range subscription with an 800ms cooldown.

Provenance. The arithmetic for labels, calibration, the ensemble, CPCV, the deflated-Sharpe gate, fusion weights, the Hawkes decay, consensus weights/thresholds, and the worker-loop set was transcribed from the live source (app/ml, app/ai, app/worker.py) and the 2026-06-01 training results. Statutory NSE charge rates, the ATR SL/TP multipliers, and the charting pane constants follow the established implementation; the SL/TP multipliers in particular are the documented post-processing defaults and should be reconfirmed against _post_process if exact values matter. This document supersedes the v1.0 architecture reference.